

Manual de Instalación y Configuración ROS2 Humble en Ubuntu para Brazo Robótico

Brayan Dayani Peña Macana

14 de diciembre de 2025

Índice general

Introducción	3
Requisitos previos	3
1. Instalación de ROS2 Humble	4
1.1. Establecer Configuración Regional	4
1.2. Configuración de Fuentes	4
1.3. Instalación del Repositorio ROS	4
1.4. Instalación de Paquetes ROS2 Humble	5
1.4.1. Corregir error con MoveIt	5
1.4.2. Actualizar el sistema	5
1.4.3. Instalar MoveIt y herramientas básicas	5
1.4.4. Instalar controladores	5
1.5. Agregar ROS2 al bashrc	6
1.5.1. Verificar la instalación	6
2. Instalación de Visual Studio Code	7
2.1. Descarga e instalación	7
2.2. Instalación de extensiones necesarias	7
3. Administración de Paquetes en ROS2	8
3.1. Concepto de paquetes	8
3.1.1. Estructura típica de un paquete	8
4. Creación de Espacio de Trabajo	9
4.1. Crear carpeta de trabajo	9
4.2. Construir el espacio de trabajo	9
5. Clonar Repositorios y Dependencias	10
5.1. Clonar el proyecto del brazo robótico	10
5.2. Instalar dependencias del sistema	10
5.2.1. Descripción de dependencias	10
5.3. Instalar librerías de Python	10
5.3.1. Descripción de librerías Python	11
6. Instalación de Ngrok	12
6.1. Descarga e instalación	12
6.2. Configuración de cuenta	12

7. Compilación del Proyecto	13
7.1. Compilar el workspace	13
7.2. Compilación selectiva (opcional)	13
7.3. Sourcing del workspace	13
8. Lanzamiento del Robot	14
8.1. Preparar el entorno	14
8.2. Lanzar el robot real	14
8.3. Verificar nodos activos	14
9. Librería para Motores Dynamixel	15
9.1. Repositorio recomendado	15
10.Herramientas Adicionales Recomendadas	16
10.1. Emuladores de terminal	16
10.1.1. Terminator	16
10.1.2. Tilix	16
10.2. Herramientas de visualización	16
11.Solución de Problemas Comunes	17
11.1. Error al compilar con colcon	17
11.2. Comando ros2 no encontrado	17
11.3. Errores con librerías de Python	17
11.4. Puerto serial ocupado	18
Próximos pasos	18

Introducción

Este manual proporciona una guía completa para la instalación y configuración del entorno de desarrollo necesario para el control del brazo robótico con ROS2 Humble en Ubuntu.

Con el fin de facilitar el proceso de instalación y ofrecer una referencia visual complementaria, se ha preparado un video explicativo que muestra paso a paso la configuración del entorno de trabajo y la preparación inicial del sistema.

- Video guía de instalación del entorno de desarrollo

Requisitos previos

- Sistema operativo: Ubuntu 22.04 LTS (Jammy Jellyfish)
- Conexión a internet estable
- Permisos de administrador (sudo)
- Espacio en disco: mínimo 10 GB disponibles
- RAM: mínimo 8 GB (recomendado 16 GB o más)

Nota Importante

Se recomienda realizar una copia de seguridad de su sistema antes de comenzar la instalación.

Capítulo 1

Instalación de ROS2 Humble

ROS2 Humble Hawksbill es la versión LTS (Long Term Support) recomendada para proyectos de robótica en Ubuntu 22.04.

1.1. Establecer Configuración Regional

Es necesario configurar el idioma y la codificación correcta del sistema:

```
sudo apt update && sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

Consejo

Agregue la última línea (`export LANG=en_US.UTF-8`) al archivo `~/.bashrc` para que se aplique automáticamente en cada sesión.

1.2. Configuración de Fuentes

Habilite el repositorio universe de Ubuntu:

```
sudo apt install software-properties-common
sudo add-apt-repository universe
```

1.3. Instalación del Repositorio ROS

Descargue e instale el repositorio oficial de ROS2:

Comandos de instalación

```
sudo apt update && sudo apt install curl -y
export ROS_APT_SOURCE_VERSION=$(curl -s
https://api.github.com/repos/ros-infrastructure/ros-apt-source/releases/
latest | grep -F "tag_name" | awk -F'{'{print $4}')
curl -L -o /tmp/ros2-apt-source.deb
"https://github.com/ros-infrastructure/ros-apt-source/releases/download/
${ROS_APT_SOURCE_VERSION}/ros2-apt-source_${ROS_APT_SOURCE_VERSION}.${
/etc/os-release && echo $VERSION_CODENAME)_all.deb"
sudo dpkg -i /tmp/ros2-apt-source.deb
```

1.4. Instalación de Paquetes ROS2 Humble

1.4.1. Corregir error con MoveIt

Primero, corrija un error conocido con MoveIt agregando esta variable de entorno:

```
# Agregar al final del archivo ~/.bashrc
export LC_NUMERIC=en_US.UTF-8
```

1.4.2. Actualizar el sistema

```
sudo apt update
sudo apt upgrade
```

1.4.3. Instalar MoveIt y herramientas básicas

```
# Instalación de MoveIt (planificación de movimientos)
sudo apt install ros-humble-moveit*

# Instalación de ROS2 Desktop y herramientas de desarrollo
sudo apt install ros-humble-desktop ros-dev-tools

# Instalación de Colcon (sistema de compilación)
sudo apt install python3-colcon-common-extensions
```

1.4.4. Instalar controladores

```
sudo apt install ros-humble-ros2-control ros-humble-ros2-
controllers
```

Nota Importante

La instalación completa de ROS2 Humble con MoveIt puede tardar entre 15 y 30 minutos, dependiendo de la velocidad de su conexión a internet.

1.5. Agregar ROS2 al bashrc

Para que ROS2 se cargue automáticamente en cada terminal nueva:

Configuración del entorno ROS 2

```
echo " Source ROS2 Humble, as instructed in
https://ros2-tutorial.readthedocs.io»> /.bashrc
echo "source /opt/ros/humble/setup.bash»> /.bashrc
source /.bashrc
```

1.5.1. Verificar la instalación

Compruebe que ROS2 esté correctamente instalado:

```
ros2 --version
```

Debería mostrar algo similar a: `ros2 cli version 0.18.x`

Capítulo 2

Instalación de Visual Studio Code

Visual Studio Code es el entorno de desarrollo integrado (IDE) recomendado para trabajar con ROS2.

2.1. Descarga e instalación

1. Visite: <https://code.visualstudio.com/>
2. Descargue el archivo `.deb` correspondiente a Ubuntu
3. Haga click derecho sobre el archivo descargado
4. Seleccione **“Abrir con Instalador de Software”**
5. Haga click en **“Instalar”**

2.2. Instalación de extensiones necesarias

Instale las siguientes extensiones desde el marketplace de VS Code:

- **C/C++** (Microsoft) - Para desarrollo en C++
- **CMake Tools** - Para gestión de proyectos CMake
- **XML Tools** - Para editar archivos URDF y launch
- **ROS** (Microsoft) - Soporte específico para ROS/ROS2

Consejo

Puede instalar extensiones desde VS Code presionando **Ctrl+Shift+X** y buscando cada una por nombre.

Capítulo 3

Administración de Paquetes en ROS2

3.1. Concepto de paquetes

ROS2 organiza el código en **paquetes** que contienen nodos y otros archivos necesarios. Cada paquete debe encargarse de una funcionalidad específica del robot.

3.1.1. Estructura típica de un paquete

- `package.xml` - Metadatos del paquete
- `CMakeLists.txt` o `setup.py` - Sistema de compilación
- `src/` - Código fuente
- `include/` - Archivos de cabecera (C++)
- `launch/` - Archivos de lanzamiento
- `config/` - Archivos de configuración
- `urdf/` - Modelos del robot

Capítulo 4

Creación de Espacio de Trabajo

Un workspace (espacio de trabajo) es el directorio donde se organizan todos los paquetes de su proyecto.

4.1. Crear carpeta de trabajo

```
mkdir -p ~/ros2_ws/src  
cd ~/ros2_ws
```

- `ros2_ws` - Nombre del workspace (puede ser cualquier nombre)
- `src` - Carpeta donde se colocan los paquetes

4.2. Construir el espacio de trabajo

```
colcon build
```

Este comando genera las siguientes carpetas:

- `build/` - Archivos intermedios de compilación
- `install/` - Archivos ejecutables y configuración
- `log/` - Registros de compilación

Nota Importante

Debe ejecutar `colcon build` cada vez que realice cambios en el código fuente para que los cambios surtan efecto.

Capítulo 5

Clonar Repositorios y Dependencias

5.1. Clonar el proyecto del brazo robótico

```
cd ~/ros2_ws/src
git clone https://github.com/lazuforever/control-architecture
-for-robotic-arm.git
```

5.2. Instalar dependencias del sistema

Instale las bibliotecas necesarias para la comunicación serial y Python:

```
sudo apt-get update && sudo apt-get install -y \
libserial-dev \
python3-pip
```

5.2.1. Descripción de dependencias

- libserial-dev - Biblioteca para comunicación serial con ESP32
- python3-pip - Gestor de paquetes de Python

5.3. Instalar librerías de Python

Instale los paquetes de Python necesarios para el proyecto:

```
pip install pyserial
pip install flask
pip install flask-ask-sdk
pip install ask-sdk
pip install mediapipe
```

5.3.1. Descripción de librerías Python

- `pyserial` - Comunicación serial con la ESP32
- `flask` - Servidor web para recibir peticiones de Alexa
- `flask-ask-sdk` - Integración de Flask con Alexa Skills Kit
- `ask-sdk` - SDK oficial de Alexa
- `mediapipe` - Biblioteca de Google para detección de manos

Advertencia

Asegúrese de tener una conexión a internet estable durante la instalación de las librerías de Python, ya que se descargarán varios paquetes grandes.

Capítulo 6

Instalación de Ngrok

Ngrok es una herramienta que crea un túnel seguro entre su computador local y una URL pública, permitiendo que Alexa se comunique con su sistema.

6.1. Descarga e instalación

1. Visite: <https://dashboard.ngrok.com/get-started/setup/linux>
2. Descargue el archivo correspondiente para Linux
3. Descomprima el archivo:

```
unzip ngrok-v3-stable-linux-amd64.zip
```

4. Mueva el ejecutable a un directorio en el PATH:

```
sudo mv ngrok /usr/local/bin/
```

5. Verifique la instalación:

```
ngrok version
```

6.2. Configuración de cuenta

1. Cree una cuenta gratuita en <https://ngrok.com>
2. Obtenga su token de autenticación desde el dashboard
3. Configure el token:

```
ngrok config add-authtoken YOUR_TOKEN_HERE
```

Consejo

La cuenta gratuita de ngrok permite crear un túnel activo a la vez, suficiente para este proyecto.

Capítulo 7

Compilación del Proyecto

Una vez clonado el repositorio e instaladas todas las dependencias, compile el proyecto completo.

7.1. Compilar el workspace

```
cd ~/ros2_ws  
colcon build
```

7.2. Compilación selectiva (opcional)

Si solo desea compilar paquetes específicos:

```
# Compilar solo un paquete  
colcon build --packages-select nombre_paquete  
  
# Compilar desde un paquete en adelante  
colcon build --packages-up-to nombre_paquete
```

7.3. Sourcing del workspace

Después de compilar, active el workspace:

```
source ~/ros2_ws/install/setup.bash
```

Nota Importante

Debe ejecutar el comando **source** en cada terminal nueva donde desee usar el proyecto. Alternativamente, puede agregarlo a su `~/ .bashrc`.

Capítulo 8

Lanzamiento del Robot

8.1. Preparar el entorno

En cada terminal nueva, ejecute:

```
cd ~/ros2_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
```

8.2. Lanzar el robot real

Para iniciar todo el sistema del brazo robótico:

```
ros2 launch rapling_bringup real_robot.launch.py
```

Este comando inicia:

- El controlador del robot
- MoveIt 2 para planificación
- El sistema de visión artificial
- La interfaz con Alexa

8.3. Verificar nodos activos

En otra terminal, verifique que los nodos estén ejecutándose:

```
ros2 node list
```

Capítulo 9

Librería para Motores Dynamixel

Si su brazo robótico utiliza servomotores Dynamixel AX-12A, puede necesitar bibliotecas adicionales.

9.1. Repositorio recomendado

- <https://github.com/jumejume1/AX-12A-servo-library/tree/master>

Consejo

Este repositorio proporciona funciones de bajo nivel para controlar motores Dynamixel. Consulte la documentación del repositorio para instrucciones de instalación específicas.

Capítulo 10

Herramientas Adicionales Recomendadas

10.1. Emuladores de terminal

Para mejorar la productividad, considere usar emuladores de terminal avanzados:

10.1.1. Terminator

Permite dividir la ventana en múltiples terminales:

```
sudo apt install terminator
```

10.1.2. Tilix

Alternativa moderna con soporte de pestañas y división:

```
sudo apt install tilix
```

10.2. Herramientas de visualización

- **RViz2** - Visualizador 3D (incluido en ROS2 Desktop)
- **rqt** - Suite de herramientas gráficas
- **PlotJuggler** - Para visualizar datos en tiempo real

Capítulo 11

Solución de Problemas Comunes

11.1. Error al compilar con colcon

Problema: Error de permisos o dependencias faltantes

Solución:

```
# Limpiar compilación anterior
rm -rf build/ install/ log/

# Verificar dependencias
rosdep install --from-paths src --ignore-src -r -y

# Compilar nuevamente
colcon build
```

11.2. Comando ros2 no encontrado

Problema: El sistema no reconoce los comandos de ROS2

Solución:

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
```

Verifique que estas líneas estén en su ~/.bashrc.

11.3. Errores con librerías de Python

Problema: Módulos de Python no encontrados

Solución:

```
# Actualizar pip
pip install --upgrade pip

# Reinstalar librería específica
pip install --force-reinstall nombre_libreria
```

11.4. Puerto serial ocupado

Problema: No se puede abrir /dev/ttyUSB0

Solución:

```
# Verificar permisos
ls -l /dev/ttyUSB*

# Agregar usuario al grupo dialout
sudo usermod -a -G dialout $USER

# Cerrar sesin y volver a iniciar
```

Próximos pasos

1. Familiarícese con los comandos básicos de ROS2
2. Revise el código fuente de los paquetes clonados
3. Consulte el Manual de Usuario para instrucciones de operación
4. Explore el repositorio en github: Repositorio